Clase 7. Regresión lineal con Python 🐍

🏃‍♀️ Ejercicio 1. ¿Podemos predecir cuánto tardará una persona en correr 5 km?

🎯 ¿Qué vamos a aprender?

Ya aprendimos a calcular una regresión lineal utilizando fórmulas matemáticas. Hicimos operaciones, encontramos la pendiente y calculamos el punto donde nuestra línea cruza el eje vertical.

¡Ahora vamos a hacer lo mismo, pero con Python!

La buena noticia es que **no tendremos que hacer todas esas operaciones a mano**. Python puede ayudarnos a encontrar la línea que mejor representa nuestros datos.

Pero antes de construir nuestro modelo, necesitamos conocer la información con la que vamos a trabajar.

🏃‍♀️ Conozcamos nuestro problema

Imaginemos que estamos organizando una carrera de 5 kilómetros.

Tenemos información de 60 corredores y conocemos dos cosas de cada uno:

- **Cuántos kilómetros entrenan a la semana.**
- **Cuántos minutos tardan en completar 5 km.**

Y nos hacemos una pregunta:

¿Las personas que entrenan más kilómetros suelen terminar la carrera en menos tiempo?

Probablemente sí, pero no siempre.

Por ejemplo, alguien que entrena 20 kilómetros a la semana podría ser más lento que alguien que entrena 15. ¿Por qué? Porque también influyen la experiencia, la condición física, el descanso y muchas otras cosas.

Sin embargo, si observamos suficientes corredores, tal vez podamos encontrar un patrón.

¡Eso es lo que vamos a investigar! 🔎

---

1. 📂 Vamos a abrir nuestros datos

Antes de hacer cualquier análisis, necesitamos cargar la información.

Piensa en cuando vas a preparar una receta: lo primero es sacar los ingredientes y revisar qué tienes.

En ciencia de datos hacemos algo parecido. Antes de construir un modelo, **primero necesitamos conocer nuestros datos**.

Para eso vamos a utilizar una biblioteca de Python llamada **Pandas**.

Pandas nos permite trabajar con información organizada en tablas, muy parecido a como lo hacemos en Excel.

🐍 Manos a Python

```
None
import pandas as pd

datos = pd.read_csv("5k.csv")
```

🔎 ¿Qué acabamos de hacer?

Vamos por partes:

`import pandas as pd`

Le estamos diciendo a Python que queremos utilizar Pandas.

El nombre `pd` es simplemente una abreviación para no tener que escribir `pandas` cada vez.

`pd.read_csv("5k.csv")`

Le pedimos a Pandas que abra nuestro archivo llamado `5k.csv`.

`datos = ...`

Guardamos la información en una variable llamada `datos`, para poder utilizarla después.

¡Listo! Ya tenemos nuestra información cargada en Python.

---

2. 👀 Vamos a echarle un vistazo

Ahora que tenemos nuestros datos, queremos ver cómo están organizados.

No necesitamos revisar las 60 filas una por una. Podemos empezar viendo solamente las primeras cinco.

```
None
datos.head()
```

	km_semanales	tiempo_5k_min
0	6	42.7
1	28	26.4
2	24	27.9
3	18	29.9
4	17	33.9

¿Qué significa `head()`?

En inglés, *head* significa cabeza.

Podemos imaginar que le estamos diciendo a Pandas:

«Enséñame la cabecita de mi tabla, quiero ver cómo empieza». 😁

Por defecto, `head()` muestra las primeras cinco filas.

Si quisiéramos ver las primeras diez, escribiríamos:

```
None
datos.head(10)
```

¿Qué estamos viendo?

Nuestro archivo tiene dos columnas:

`km_semanales`: la cantidad de kilómetros que entrena cada corredor durante una semana.

`tiempo_5k_min`: los minutos que tarda ese corredor en completar una carrera de 5 km.

Cada fila representa a una persona diferente.

Y aquí tenemos algo muy importante: **cada corredor tiene un par de valores que están relacionados entre sí**.

Esos pares de valores serán la base de nuestra regresión lineal.

---

3. 📊 ¿Qué tan grande es nuestro dataset?

Ahora queremos saber cuánta información tenemos.

```
None
datos.shape
```

¿Qué hace `shape`?

Shape significa forma.

Nos dice cuántas filas y columnas tiene nuestra tabla.

En nuestro caso, el resultado será:

```
None
(60, 2)
```

Esto significa que tenemos:

- **60 filas:** una por cada corredor.
- **2 columnas:** kilómetros semanales y tiempo de carrera.

¡Tenemos 60 corredores para investigar! 🏃

---

4. 🔍 Conozcamos un poquito mejor nuestros datos

Hasta ahora sabemos qué columnas tenemos, pero todavía no conocemos sus valores generales.

Por ejemplo:

- ¿Cuántos kilómetros entrena en promedio nuestro grupo?
- ¿Cuál es el tiempo promedio para completar los 5 km?
- ¿Quién entrena más kilómetros?
- ¿Cuál es el tiempo más rápido?

Podríamos revisar los 60 registros y hacer las cuentas a mano, pero Pandas tiene una forma mucho más sencilla.

None

```
datos.describe()
```

¿Qué hace `describe()`?

Le pedimos a Pandas que nos haga un pequeño resumen estadístico de nuestros datos.

¡Y aquí vamos a recuperar cosas que ya aprendimos en clases anteriores! 💜

	km_semanales	tiempo_5k_min
count	60.000000	60.000000
mean	20.300000	30.883333
std	8.836174	4.365705
min	5.000000	24.400000
25%	14.250000	27.575000
50%	20.500000	30.050000
75%	28.000000	33.825000
max	35.000000	42.700000

Entre los resultados encontraremos:

Resultado	¿Qué significa?
`count`	Cuántos valores tenemos
`mean`	El promedio
`std`	La desviación estándar
`min`	El valor más pequeño
`25%`	El 25% de los valores está en ese número o por debajo
`50%`	La mediana
`75%`	El 75% de los valores está en ese número o por debajo
`max`	El valor más grande

Imagina que encontramos que el tiempo promedio de los corredores es de 31 minutos.

¿Eso significa que todos tardan aproximadamente 31 minutos?

¡No necesariamente!

Algunos pueden terminar mucho más rápido y otros necesitar más tiempo.

Por eso es importante revisar no solamente el promedio, sino también cómo se distribuyen nuestros datos.

Recuerda: un promedio nos ayuda a resumir la información, pero no nos cuenta toda la historia.

---

5. 🧹 ¿Tenemos información incompleta?

Antes de construir un modelo, también necesitamos revisar si faltan datos.

Imagina que tenemos el registro de un corredor, pero olvidamos anotar cuánto tardó en completar los 5 km.

¿Cómo podríamos utilizarlo para aprender la relación entre entrenamiento y tiempo de carrera?

Primero tenemos que identificar si existen esos espacios vacíos.

```
None
datos.isnull().sum()
```

¿Qué acabamos de hacer?

`isnull()` revisa si hay valores vacíos en nuestra tabla.

`sum()` cuenta cuántos encontró en cada columna.

En nuestro dataset esperamos obtener:

```
None
km_semanales    0
tiempo_5k_min   0
```

¡Buenas noticias! 🎉

No tenemos valores faltantes, así que podemos continuar.

En proyectos reales no siempre tendremos tanta suerte. A veces tendremos que limpiar, corregir o completar información antes de construir un modelo.

🧠 Hagamos una pequeña pausa para pensar

Ya conocemos nuestros datos, pero todavía no hemos construido ningún modelo de Machine Learning.

Y eso está perfecto.

Una parte muy importante de la ciencia de datos consiste en entender la información antes de intentar hacer predicciones.

Hasta ahora sabemos que tenemos 60 corredores, conocemos las variables disponibles, revisamos algunos estadísticos y comprobamos que no hay valores vacíos.

Pero todavía nos falta descubrir algo:

¿Realmente existe una relación entre los kilómetros que entrenan y el tiempo que tardan en correr?

Para responderlo, en la siguiente sección vamos a convertir nuestros datos en una gráfica.

¡Es hora de ver qué nos cuentan esos 60 corredores! 📈

6. 📈 Vamos a visualizar nuestros datos

Ya sabemos cuántos corredores tenemos, cuánto entrenan y cuánto tardan en completar los 5 km.

Pero ahora queremos responder una pregunta:

¿Las personas que entrenan más kilómetros a la semana suelen terminar la carrera en menos tiempo?

Podríamos revisar los 60 registros uno por uno, pero sería complicado encontrar un patrón.

¡Mejor vamos a dibujarlos!

Para eso utilizaremos una **gráfica de dispersión**, que ya conocimos en nuestras clases de visualización de datos.

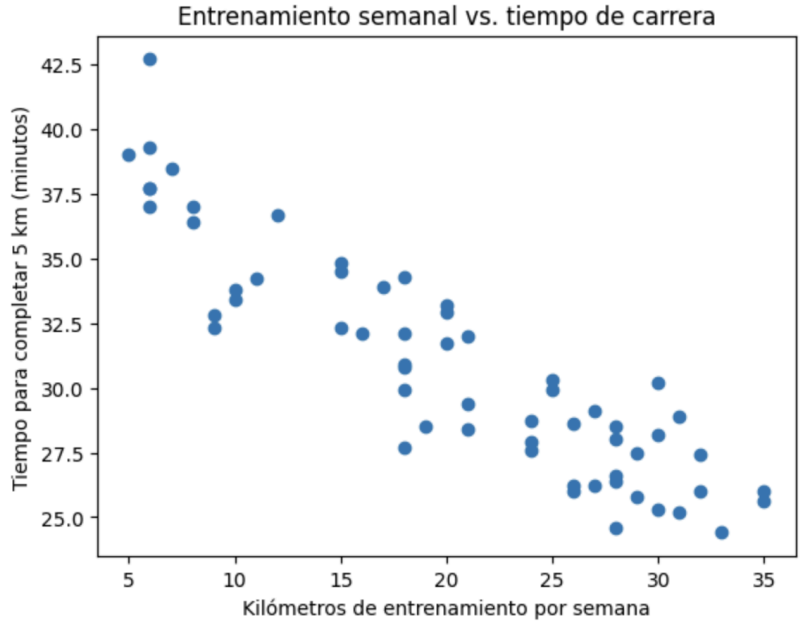
🐍 Manos a Python

```python
import matplotlib.pyplot as plt
```

```
plt.scatter(datos["km_semanales"], datos["tiempo_5k_min"])

plt.xlabel("Kilómetros de entrenamiento por semana")
plt.ylabel("Tiempo para completar 5 km (minutos)")
plt.title("Entrenamiento semanal vs. tiempo de carrera")

plt.show()
```



🔎 ¿Qué acabamos de hacer?

`import matplotlib.pyplot as plt`

Importamos Matplotlib, la biblioteca que utilizamos para crear gráficas en Python.

`plt.scatter()`

Creamos una gráfica de dispersión.

Cada puntito representa a uno de nuestros 60 corredores.

`plt.xlabel()` y **`plt.ylabel()`**

Agregamos nombres a los ejes para saber qué estamos observando.

`plt.title()`

Le ponemos un título a nuestra gráfica.

```
plt.show()
```

Le pedimos a Python que nos muestre el resultado.

🤔 ¿Qué nos está diciendo la gráfica?

Observa los puntitos.

¿Notas que, en general, conforme aumentan los kilómetros de entrenamiento, disminuye el tiempo de carrera?

¡Eso significa que parece existir una relación entre nuestras dos variables!

Pero también notarás que los puntos no forman una línea perfecta.

Y tiene sentido: **dos personas que entrenan la misma cantidad de kilómetros no necesariamente corren a la misma velocidad**.

Eso es precisamente lo interesante de trabajar con datos reales o que simulan situaciones reales.

---

7. 🧩 ¿Cuáles son nuestras variables X y Y?

Antes de construir el modelo, necesitamos decidir qué información vamos a utilizar para hacer nuestra predicción.

Recordemos nuestra fórmula de regresión lineal:

Y = a + bX

Donde:

- **X:** la información que utilizamos para hacer una predicción.
- **Y:** lo que queremos predecir.
- **a:** el intercepto, donde nuestra línea cruza el eje vertical.
- **b:** la pendiente, que nos dice cuánto cambia Y cuando X aumenta una unidad.

En nuestro ejemplo:

X = kilómetros de entrenamiento por semana.

Y = tiempo para completar los 5 km.

Es decir, queremos utilizar los kilómetros de entrenamiento para estimar el tiempo de carrera.

🐍 Manos a Python

```python
X = datos[["km_semanales"]]
y = datos["tiempo_5k_min"]
```

🔎 ¿Qué acabamos de hacer?

`X = datos[["km_semanales"]]`

Seleccionamos la columna que utilizaremos para hacer las predicciones.

`y = datos["tiempo_5k_min"]`

Seleccionamos la columna que queremos predecir.

👀 ¿Por qué X tiene dos pares de corchetes?

¡Buena pregunta!

Cuando escribimos:

```python
datos["km_semanales"]
```

Pandas nos devuelve una **Serie**, es decir, una sola columna de valores.

Pero cuando escribimos:

```python
datos[["km_semanales"]]
```

Nos devuelve un **DataFrame**, una pequeña tabla.

Scikit-learn, la biblioteca que utilizaremos para entrenar nuestro modelo, espera recibir las variables de entrada en forma de tabla, aunque solamente tengamos una columna.

Por eso utilizamos los corchetes dobles para X.

---

8. 🤖 ¡Vamos a construir nuestro primer modelo!

Ahora sí, llegó el momento de utilizar Machine Learning.

Ya tenemos nuestros datos, los exploramos, hicimos una gráfica y elegimos nuestras variables X y Y.

Lo que sigue es pedirle a Python que encuentre la línea que mejor se ajuste a nuestros corredores.

Para eso utilizaremos una biblioteca llamada **Scikit-learn**, que contiene herramientas para construir modelos de Machine Learning.

🐍 Manos a Python

```python
from sklearn.linear_model import LinearRegression

modelo = LinearRegression()

modelo.fit(X, y)
```

🔎 ¿Qué acabamos de hacer?

Vamos paso a paso.

```python
from sklearn.linear_model import LinearRegression
```

Importamos la herramienta que sabe calcular una regresión lineal.

```python
modelo = LinearRegression()
```

Creamos nuestro modelo.

Es como sacar una calculadora nueva de su caja: ya tenemos la herramienta, pero todavía no le hemos dado los datos con los que va a trabajar.

```python
modelo.fit(X, y)
```

¡Aquí sucede lo importante!

La instrucción `fit()` le dice al modelo:

«Aquí están los kilómetros que entrenaron estas personas y los tiempos que obtuvieron. Encuentra la línea que mejor represente su relación».

A este proceso le llamamos **entrenar un modelo**.

En este caso, entrenar significa encontrar los valores de la pendiente y el intercepto que hacen que nuestra línea se acerque lo más posible a los datos.

¡Y Python acaba de hacer las operaciones matemáticas por nosotros! 🎉

---

9. 📐 ¿Qué pendiente e intercepto encontró Python?

Recordemos que nuestra fórmula es:

Y = a + bX

Ahora queremos descubrir qué valores encontró Python para **a** y **b**.

🐍 Manos a Python

```python
print("Pendiente:", modelo.coef_[0])
print("Intercepto:", modelo.intercept_)
```

```
Pendiente: -0.4461424912082665
Intercepto: 39.940025904861145
```

🔎 ¿Qué significan estos resultados?

`modelo.coef_[0]`

Nos muestra la pendiente de nuestra línea.

En este ejercicio esperamos que sea un número negativo.

¿Por qué?

Porque, en general, cuando aumentan los kilómetros de entrenamiento, disminuyen los minutos necesarios para completar los 5 km.

Por ejemplo, si nuestra pendiente fuera **-0.45**, significaría que por cada kilómetro adicional de entrenamiento semanal, el tiempo estimado de carrera disminuye 0.45 minutos.

Eso equivale a aproximadamente 27 segundos.

¡Ojo! Esto describe la relación que encontró nuestro modelo, no significa que cualquier persona pueda mejorar exactamente 27 segundos con solo entrenar un kilómetro más.

`modelo.intercept_`

Nos muestra el punto donde nuestra línea cruza el eje Y.

En nuestro ejemplo, representa el tiempo que el modelo estimaría para alguien que entrena cero kilómetros a la semana.

Pero hay un detalle: nuestros datos no incluyen corredores que entrenen cero kilómetros.

Por eso, aunque Python pueda calcular ese valor, **no necesariamente sería una predicción confiable**.

🧠 Conectemos con lo que ya aprendimos

¿Recuerdas cuando calculamos la pendiente y el intercepto utilizando fórmulas matemáticas?

¡Acabamos de hacer exactamente lo mismo!

La diferencia es que ahora Python hizo las operaciones utilizando los datos de nuestros 60 corredores.

Las matemáticas siguen ahí. Simplemente tenemos una herramienta que nos ayuda a hacerlas más rápido. 💜

10. 📈 ¡Vamos a dibujar nuestra línea de regresión!

Ya tenemos nuestro modelo entrenado y conocemos su pendiente e intercepto.

Pero hasta ahora solamente hemos visto números.

¿Y si pudiéramos ver la línea que encontró Python?

Recordemos nuestra gráfica de dispersión: cada puntito representa a un corredor, pero los puntos están repartidos por toda la gráfica.

Nuestra línea de regresión intentará representar la tendencia general de esos puntos.

🐍 Manos a Python

Primero vamos a pedirle al modelo que calcule el tiempo estimado para cada uno de nuestros corredores.

```
None
predicciones = modelo.predict(X)
```

🔎 ¿Qué acabamos de hacer?

`modelo.predict(X)`

Le estamos diciendo a nuestro modelo:

«Ya aprendiste la relación entre kilómetros de entrenamiento y tiempo de carrera. Ahora utiliza esa relación para calcular cuánto tiempo estimarías para cada corredor».

El modelo toma los kilómetros semanales de cada persona, los sustituye en la ecuación de regresión lineal y calcula un tiempo estimado.

Importante: estos resultados son predicciones del modelo, no los tiempos reales que registramos.

Ahora vamos a dibujar los resultados.

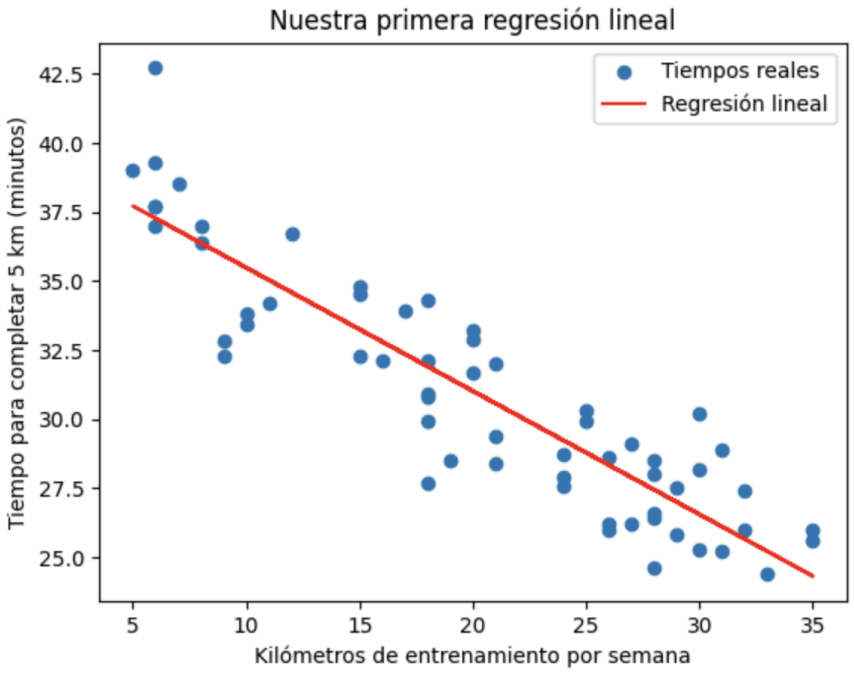
None

```
plt.scatter(X["km_semanales"], y, label="Tiempos reales")

plt.plot(
    X["km_semanales"],
    predicciones,
    color="red",
    label="Regresión lineal"
)

plt.xlabel("Kilómetros de entrenamiento por semana")
plt.ylabel("Tiempo para completar 5 km (minutos)")
plt.title("Nuestra primera regresión lineal")

plt.legend()
plt.show()
```



👀 ¿Qué estamos viendo?

Ahora tenemos dos elementos en nuestra gráfica:

- **Los puntitos:** representan los tiempos reales de nuestros 60 corredores.
- **La línea roja:** representa los tiempos que estima nuestro modelo.

¿Notas que la línea pasa por el centro de la nube de puntos?

No pasa exactamente por todos, ¡y eso está bien!

Imagina que quieres dibujar una línea con una regla sobre una hoja llena de puntitos. No puedes tocar todos al mismo tiempo, así que buscas una posición que se acerque lo mejor posible al conjunto.

Eso es lo que hace la regresión lineal.

De hecho, el modelo busca que la suma de los errores al cuadrado sea lo más pequeña posible. Es el mismo principio matemático que vimos cuando calculamos la regresión a mano.

---

11. 🔮 ¡Hagamos nuestra primera predicción!

Ahora imaginemos que llega una nueva persona y nos dice:

«Yo entreno 18 kilómetros a la semana. ¿Cuánto tiempo podría tardar en correr 5 km?»

¡Podemos utilizar nuestro modelo para estimarlo!

🐍 Manos a Python

```
None
nuevo_corredor = pd.DataFrame({
    "km_semanales": [18]
})

tiempo_estimado = modelo.predict(nuevo_corredor)

print("Tiempo estimado:", round(tiempo_estimado[0], 1),
"minutos")
```

```
Tiempo estimado: 31.9 minutos
```

🔎 ¿Qué acabamos de hacer?

`nuevo_corredor`

Creamos una pequeña tabla con la información de nuestro nuevo corredor.

Solamente conocemos una cosa: entrena 18 kilómetros a la semana.

`modelo.predict(nuevo_corredor)`

Le pedimos al modelo que utilice lo que aprendió para estimar su tiempo de carrera.

`tiempo_estimado[0]`

Tomamos la primera predicción. Como solamente estamos preguntando por una persona, tenemos un único resultado.

`round(..., 1)`

Redondeamos el tiempo a un decimal para que sea más fácil de leer.

🧠 ¿De dónde salió esa predicción?

¡De nuestra ecuación!

Recordemos:

Y = a + bX

Nuestro modelo ya encontró los valores de **a** y **b**.

Ahora solamente sustituye X por 18, porque nuestro nuevo corredor entrena 18 kilómetros semanales.

Podemos comprobarlo nosotros mismos:

None

```
pendiente = modelo.coef_[0]
intercepto = modelo.intercept_

resultado_manual = intercepto + pendiente * 18

print("Predicción con la fórmula:", round(resultado_manual, 1))
print("Predicción con Python:", round(tiempo_estimado[0], 1))
```

```
Predicción con la fórmula: 31.9
Predicción con Python: 31.9
```

¡Ambos resultados coinciden! 🎉

Python no está haciendo magia. Está utilizando la misma fórmula matemática que ya aprendimos.

---

12. 🏃‍♀️ ¿Y si queremos predecir varios corredores?

Imaginemos que tenemos tres personas nuevas:

- Una entrena 8 kilómetros a la semana.
- Otra entrena 18 kilómetros.
- Y otra entrena 30 kilómetros.

¿Tenemos que ejecutar el modelo tres veces?

¡No! Podemos hacer todas las predicciones al mismo tiempo.

🐍 Manos a Python

None

```
nuevos_corredores = pd.DataFrame({
    "km_semanales": [8, 18, 30]
})

nuevos_corredores["tiempo_estimado"] = modelo.predict(
    nuevos_corredores[["km_semanales"]]
)

nuevos_corredores
```

	km_semanales	tiempo_estimado
0	8	36.370886
1	18	31.909461
2	30	26.555751

🔎 ¿Qué acabamos de hacer?

Creamos una tabla con los kilómetros de entrenamiento de tres personas.

Después utilizamos `predict()` para calcular sus tiempos estimados.

Finalmente, guardamos las predicciones en una nueva columna llamada `tiempo_estimado`.

¡Ahora tenemos una tabla con la información original y los resultados de nuestro modelo!

🤔 Pensemos un poquito

Observa los tres tiempos estimados.

¿Qué sucede conforme aumentan los kilómetros de entrenamiento?

Seguramente notarás que los tiempos estimados disminuyen.

Eso ocurre porque nuestra línea tiene una pendiente negativa.

Pero recuerda: nuestro modelo solamente conoce los kilómetros de entrenamiento.

No sabe si una persona tiene experiencia corriendo, si durmió bien, si está lesionada o si ese día hace muchísimo calor.

Por eso, **una predicción es una estimación basada en los datos, no una garantía de lo que sucederá**.

---

13. 🎯 ¿Qué tan bueno es nuestro modelo?

Hasta ahora nuestro modelo puede hacer predicciones.

Pero nos falta responder una pregunta muy importante:

¿Qué tan cerca están sus predicciones de los tiempos reales?

Imagina que el modelo predice que alguien terminará los 5 km en 30 minutos.

Si esa persona realmente tarda 31 minutos, ¡estuvo bastante cerca!

Pero si termina en 42 minutos, la diferencia es mucho mayor.

A esa diferencia entre el valor real y el valor estimado le llamamos **error de predicción**.

🐍 Manos a Python

Vamos a calcular el error absoluto promedio, también conocido como **MAE**, por sus siglas en inglés (*Mean Absolute Error*).

```
None
from sklearn.metrics import mean_absolute_error
```

```
error_promedio = mean_absolute_error(y, predicciones)

print("Error promedio:", round(error_promedio, 2), "minutos")
```

```
Error promedio: 1.55 minutos
```

🔎 ¿Qué significa este resultado?

El MAE nos dice, en promedio, cuántos minutos se alejan nuestras predicciones de los tiempos reales.

Por ejemplo, si obtenemos un MAE de 2 minutos, significa que nuestras predicciones se equivocan aproximadamente 2 minutos en promedio.

¡Eso es mucho más fácil de interpretar que simplemente decir que el modelo es bueno o malo!

Mientras más pequeño sea el MAE, más cerca están nuestras predicciones de los valores reales.

⚠️ Un detalle importante

Aquí estamos calculando el error utilizando a los mismos corredores con los que entrenamos nuestro modelo.

Esto nos ayuda a entender cómo se ajustó a los datos que ya conoce, pero **no nos dice todavía qué tan bien funcionaría con corredores nuevos**.

Es como estudiar para un examen resolviendo los mismos ejercicios varias veces.

Puede que te salgan muy bien porque ya los conoces.

Pero ¿qué pasaría si te ponen preguntas que nunca habías visto?

En Machine Learning sucede algo parecido.

Por eso, más adelante aprenderemos a separar los datos en dos grupos: uno para entrenar y otro para poner a prueba nuestro modelo.

💜 ¿Qué aprendimos con nuestros corredores?

¡Acabamos de construir nuestro primer modelo de regresión lineal con Python!

Partimos de una tabla con información de 60 corredores y logramos:

- Encontrar una relación entre los kilómetros de entrenamiento y los tiempos de carrera.
- Dibujar la línea que representa esa relación.
- Identificar la pendiente y el intercepto.
- Utilizar la ecuación para hacer predicciones.
- Calcular qué tan lejos estuvieron las predicciones de los valores reales.

Y lo más importante: **entendimos que un modelo de Machine Learning utiliza patrones de los datos para hacer estimaciones**.

Ahora que sabemos cómo funciona, vamos a llevar este conocimiento a otro escenario.

¿Qué pasaría si, en lugar de predecir tiempos de carrera, quisiéramos ayudar a una cafetería a planear sus ventas?

☕ ¡Vamos a descubrirlo!